

Dory.md — Person 1

Backend & Integration Layer

FastAPI scaffolding · SQLite · File ingestion · 8 REST endpoints · Render deploy

Your role	Backend & Integration Engineer — owns the API surface, data persistence, and file ingestion.
What you ship	A FastAPI server that exposes 8 REST endpoints, persists chunks to SQLite, handles file uploads (Markdown/PDF), and is deployed to Render. Endpoint logic that needs intelligence (decay scoring, embeddings, search) is delegated to Person 2's modules — you call them, you don't implement them.
What you DON'T do	No embedding code. No decay formula implementation. No retrieval logic. No frontend code. If a prompt seems to require any of those, stop and ask Person 2 or 3.
Your Hour 0-42 path	FastAPI scaffolding (Hours 0-2) → SQLite + models (Hours 2-4) → File upload + parsing (Hours 4-7) → Endpoint shells (Hours 7-12) → Wire to P2 modules (Hours 12-20) → Deploy & debug (Hours 20-30) → Polish, bug bash, demo prep (Hours 30-42). Hours 42-48 are sleep/buffer — be available for last-minute coordination but no new work.
Riskiest task	Wiring P2's intelligence modules into your endpoints (Hours 12-20). Signature mismatches and import errors are the most common integration bugs. The doc shows you the exact signatures to expect and how to fall back to mocks when P2's modules aren't ready yet.
Branch strategy	Work on branch `feat/backend`. Commit every 30-60 minutes. Open a draft PR to main around Hour 8 so the rest of the team can see your structure even before it's merged.

Contents

- 1. Before You Start — Checklist
- 2. The Three-Person Integration Contract
- 3. Pre-Hackathon Setup (Do This DAYS Before)
- 4. Hours 0-2 — FastAPI Scaffolding
- 5. Hours 2-4 — SQLite Schema & Pydantic Models
- 6. Hours 4-7 — File Upload & Parsing (Markdown + PDF)
- 7. Hours 7-12 — Endpoint Shells & Routing
- 8. Hours 12-20 — Integration with Person 2's Intelligence Modules
- 9. Hours 20-30 — Deployment to Render
- 10. Hours 30-42 — Polish, Bug Bash, Demo Prep
- 11. Stretch Tasks (if you finish early)
- 12. Descope Ladder (if you fall behind)
- 13. Handoff Checklists
- 14. Troubleshooting Guide

1. Before You Start — Checklist

If any of these isn't true, do not start the hackathon — fix it first. Each takes 10 minutes and saves you 4+ hours of mid-hackathon panic.

✓ / ✗	Item	How to verify
	Python 3.11+ installed	Terminal: <code>`python --version`</code>
	Git installed and configured (name + email)	Terminal: <code>`git config --list`</code>
	GitHub account, can push to team repo	Try <code>`git push`</code> to a test branch
	Code editor with LLM (Cursor / Copilot / Claude Code / similar)	Open a project, paste a test prompt
	Render account created (no project yet)	render.com — sign up with GitHub
	Groq API key obtained (will be needed by P2 too)	console.groq.com — generate key
	Cloned the team repo locally	<code>`git clone`</code>
	Read this entire document end to end	Don't start writing code until you have

2. The Three-Person Integration Contract

The Three-Person Integration Contract

All three of you must agree on these JSON shapes BEFORE any of you start building. If any field changes during the hackathon, the person changing it must notify the other two in Slack/Discord immediately. The contract is the single source of truth — everything else flows from it.

Endpoint	Method	Owner	Returns
/api/health	GET	P1 routes, P2 logic	{categories: [{name, retention, urgency, count}]}
/api/health?time_offset_hours=N	GET	P1 routes, P2 logic	Same shape, computed at t+N
/api/ingest	POST	P1 fully	{status, chunks_count, doc_id}
/api/search	POST	P1 routes, P2 logic	{results: [{chunk_id, content, score, retention}]}
/api/discovery	GET	P1 routes, P2 logic	{has_discovery, chunk?, reason?}
/api/review/{chunk_id}	POST	P1 fully	{status, new_access_count}
/api/quiz/start	POST	P1 routes, P2 logic	{quiz_id, questions: [{q, options, correct}]}
/api/quiz/submit	POST	P1 fully	{score, correct_count}

The Chunk JSON Shape

This is the canonical shape of a Chunk object. Person 1 stores it, Person 2 enriches and retrieves it, Person 3 displays it. If you need a field not listed here, propose the addition in chat before adding it.

```
{ "id": "chk_8a3f...", // UUID, generated by P1
  "content": "...chunk text 200-500 chars...", // P1 splits source
  "source_type": "file", // P1 sets at ingest
  "source_name": "research_notes.md", // filename
  "category": "technical" | "personal" | ..., // P2 sets via Groq
  "created_at": "2026-04-25T10:00:00Z", // ISO 8601
  "last_accessed": "2026-04-25T10:00:00Z", // updated on review
  "access_count": 0, // increments on review
  "stability_S": 7.0, // P2 decay parameter
  "complexity_k": 1.0 // P2 decay parameter
}
```

Your contracts with Person 2 (Intelligence)

You will import Python modules from Person 2's branch. Until those modules exist, you'll mock them with stub functions. The interfaces:

P2 Module	Function	What it does	Your stub returns
intelligence/decay.py	compute_retention(chunks, time_offset_hours=0)	NumPy vectorized decay computation	Random floats 0-1 for now
intelligence/decay.py	aggregate_by_category(chunks, retentions)	Groups retention values by category	Hardcoded fake categories
intelligence/embedder.py	embed_chunks(texts: list) -> np.ndarray	Returns 384-dim embeddings	Random 384-dim arrays
intelligence/search.py	hybrid_search(query, top_k=10)	BM25 + dense + RRF	Returns first N chunks unfiltered
intelligence/classifier.py	classify_chunks(chunks: list)	Groq LLM categorization	Returns 'general' for everything
intelligence/quiz.py	generate_quiz(chunk_ids: list)	5 MCQs from Groq	Returns 5 fake questions

Your contract with Person 3 (Frontend)

Person 3 builds against your endpoints. They will be running their dev server on localhost:5173 while you run on localhost:8000. Your job:

- Add CORS middleware allowing http://localhost:5173 from Hour 0
- When deploying, also allow Person 3's Vercel URL (they give you the URL Hour 24)
- Return JSON in EXACTLY the shapes in the table above. No extra fields, no missing fields.
- Use HTTP status codes correctly: 200 OK, 201 Created, 400 bad request, 404 not found, 500 server error

3. Pre-Hackathon Setup (Do This DAYS Before)

These tasks cannot be done during the 48 hours. Each one is BLOCKING. Do them in the week before the hackathon, not the night before.

3.1 — Create Render account & warm it up

STOP — HUMAN STEP REQUIRED

Steps:

1. Go to **render.com**, sign up using GitHub
2. Connect your GitHub account
3. (Don't create a service yet — you'll do that at Hour 20 when the code is ready)
4. Verify your email so the account is fully active
5. Read Render's free tier limits: 750 hours/month free, sleeps after 15 min of inactivity, ~30 sec cold start. **This means your demo backend will be cold by judging time — see Hour 30 for the keep-warm trick.**

3.2 — Get a Groq API key

STOP — HUMAN STEP REQUIRED

1. Go to **console.groq.com**, sign up (free tier is generous)
2. Generate an API key in the dashboard
3. Save it as an environment variable: `export GROQ_API_KEY='gsk_...'`
4. Add it to your local `.env` file (which is gitignored — never commit keys)
5. Share with Person 2 too — they need it for classification + quiz

3.3 — Set up the team repo

PROMPT TO PASTE INTO YOUR LLM

Create a new directory called dory-md. Inside it, set up a Python project with the following structure, ready for FastAPI development:

```
dory-md/  
  ■■■ backend/  
  ■ ■■■ __init__.py  
  ■ ■■■ main.py # FastAPI app entrypoint  
  ■ ■■■ routes/ # API route handlers  
  ■ ■■■ models/ # Pydantic and SQLAlchemy models  
  ■ ■■■ services/ # Business logic  
  ■ ■■■ integrations/ # File parsing helpers  
  ■ ■■■ tests/ # pytest tests  
  ■■■ intelligence/ # Person 2's territory  
  ■ ■■■ __init__.py # Empty for now  
  ■■■ frontend/ # Person 3's territory  
  ■■■ .env.example  
  ■■■ .gitignore # Python + Node defaults, INCLUDE .env  
  ■■■ requirements.txt  
  ■■■ README.md
```

Initialize as a git repo. Create requirements.txt with: fastapi, uvicorn[standard], pydantic, sqlalchemy, aiosqlite, python-multipart, pypdf, python-dotenv, pytest, pytest-asyncio.

Create .gitignore with Python defaults plus: .env, *.db, __pycache__/, node_modules/, .venv/, dory.db.

Create .env.example with placeholders: GROQ_API_KEY=, DATABASE_URL=sqlite+aiosqlite:///./dory.db.

Do not write any application code yet. Just create the structure and the empty/placeholder files. After creating, show me the directory tree.

After the LLM completes this prompt, push to GitHub and share the repo URL with Person 2 and Person 3. They'll branch from main.

4. Hours 0-2 — FastAPI Scaffolding

Goal: a runnable FastAPI server with health check and CORS configured.

Prompt 1 — Bootstrap the FastAPI app

PROMPT TO PASTE INTO YOUR LLM

Inside backend/main.py, create a minimal FastAPI application that:

1. Loads environment variables from .env using python-dotenv at startup.
2. Initializes the FastAPI app with title='Dory.md API', version='0.1.0'.
3. Adds CORS middleware that allows origins: http://localhost:5173 and http://localhost:3000. Allow all methods, all headers, allow_credentials=True.
4. Adds a single GET endpoint at /api/health that returns {'status': 'ok', 'service': 'dory-backend'}.
5. Adds a startup event handler that prints 'Dory.md backend starting...' to confirm boot.

Keep it minimal – no database, no other endpoints yet. Just enough that `uvicorn backend.main:app --reload` runs and I can visit http://localhost:8000/api/health and see the JSON response.`

After writing, give me the exact uvicorn command to run.

Prompt 2 — Verify it runs

After the LLM finishes, run the uvicorn command in your terminal. In another terminal (or your browser), check:

- `curl http://localhost:8000/api/health` returns the JSON above
- Visit `http://localhost:8000/docs` — FastAPI's auto-generated Swagger UI loads
- If you change main.py and save, uvicorn auto-reloads (because of `--reload` flag)

Heads-up

If you get 'Address already in use' on port 8000, another process is using it. Either kill it (`lsof -ti:8000 | xargs kill -9` on macOS/Linux) or run on a different port with `--port 8001` — but if you change the port, you must tell Person 3 so they update their frontend's API base URL.

Prompt 3 — Project README

PROMPT TO PASTE INTO YOUR LLM

Update README.md at the project root with a 'Quick Start' section that shows:

1. How to clone and set up Python venv
2. How to install requirements
3. How to copy .env.example to .env and fill in keys
4. How to run the backend (uvicorn command)
5. The 9 endpoints (just a list – don't document them yet)

Keep it under 80 lines. This is for the rest of the team to onboard.

Commit before moving on

`git checkout -b feat/backend` then `git add -A && git commit -m 'feat(backend): scaffold FastAPI app with health endpoint'` then `git push -u origin feat/backend`. Open a draft PR on GitHub.

5. Hours 2-4 — SQLite Schema & Pydantic Models

Goal: a working SQLite database with Chunk, Document, and ReviewLog tables, with async CRUD operations.

Prompt 4 — Database connection setup

PROMPT TO PASTE INTO YOUR LLM

In backend/models/__init__.py, set up async SQLAlchemy with aiosqlite.

Requirements:

1. Use SQLAlchemy 2.0 style (DeclarativeBase, async session).
2. Read DATABASE_URL from environment, default to 'sqlite+aiosqlite:///./dory.db' if missing.
3. Create an async_engine and async_session_maker.
4. Define a Base class for models to inherit from.
5. Provide a dependency function get_db() that yields an AsyncSession and closes it on exit.
6. Provide an init_db() coroutine that creates all tables (uses Base.metadata.create_all).

Don't define any models yet – just the connection plumbing.

Then in backend/main.py, modify the startup event to await init_db() so tables are created on app boot.

Prompt 5 — Define the Chunk, Document, and ReviewLog tables

PROMPT TO PASTE INTO YOUR LLM

In backend/models/chunk.py, define three SQLAlchemy 2.0 models that inherit from Base (imported from backend/models/__init__.py).

Model 1 – Document:

- id: str (UUID, primary key)
- source_type: str (default 'file' – kept as a string column for forward compatibility if you add other ingestion sources later)
- source_name: str (filename)
- imported_at: datetime
- metadata_json: str (nullable, JSON string of extra metadata)

Model 2 – Chunk:

- id: str (UUID, primary key, default generated)
- document_id: str (FK to Document.id, indexed)
- content: str (the chunk text, 200-500 chars typically)
- category: str (default 'general', updated by P2 classifier later)
- created_at: datetime (default now)
- last_accessed: datetime (default now, updated on review)
- access_count: int (default 0)
- stability_S: float (default 7.0)
- complexity_k: float (default 1.0)

Model 3 – ReviewLog:

- id: int (autoincrement primary key)
- chunk_id: str (FK to Chunk.id)
- reviewed_at: datetime (default now)

Use sqlalchemy.orm.Mapped and mapped_column. Use uuid.uuid4().hex for ID defaults. Use datetime.utcnow for timestamp defaults. Add appropriate indexes (chunk.document_id, chunk.last_accessed, reviewlog.chunk_id).

Also create matching Pydantic models in `backend/models/schemas.py`:

- `ChunkOut` (matches the JSON contract: `id`, `content`, `source_type`, `source_name`, `category`, `created_at`, `last_accessed`, `access_count`, `stability_S`, `complexity_k`)
- `DocumentOut`
- `ReviewLogOut`

Use `ConfigDict(from_attributes=True)` so they can be created from SQLAlchemy objects.

Make sure these models are imported in `backend/models/__init__.py` so `init_db()` can find them via `Base.metadata`.

Prompt 6 — CRUD operations

PROMPT TO PASTE INTO YOUR LLM

In `backend/services/chunk_service.py`, write async CRUD functions for `Chunk` and `Document`.

Functions needed:

- `create_document(db, source_type, source_name) -> Document`
- `create_chunks_batch(db, document_id, chunk_texts: list[str]) -> list[Chunk]`
(creates chunks in a single transaction; uses default `S=7.0`, `k=1.0`)
- `get_chunk(db, chunk_id) -> Chunk | None`
- `get_all_chunks(db) -> list[Chunk]`
- `get_chunks_by_ids(db, ids: list[str]) -> list[Chunk]`
- `update_chunk_access(db, chunk_id) -> Chunk`
(increments `access_count`, sets `last_accessed=now`, also creates a `ReviewLog` row, multiplies `stability_S` by 1.5 to simulate review reinforcement, capped at 365)
- `update_chunk_category(db, chunk_id, category) -> Chunk`
- `count_chunks(db) -> int`

All functions are async, take `db: AsyncSession` as first arg, use proper `async/await` with `select()` statements.

After writing, also add a quick test in `backend/tests/test_chunk_service.py` that creates a doc, adds 3 chunks, retrieves them, and updates one's access count. Use `pytest-asyncio`.

Prompt 7 — Verify with a smoke test

PROMPT TO PASTE INTO YOUR LLM

Run `pytest backend/tests/test_chunk_service.py` and confirm it passes. If it fails, show me the error and we'll fix it. After it passes, run the FastAPI server again and verify that `dory.db` is created in the project root with the three tables. Use `sqlite3` command-line or `DBeaver` to inspect the schema.

Heads-up

If you get an error about `'NoneType' has no attribute 'metadata'`, you forgot to import your model files in `backend/models/__init__.py`. SQLAlchemy can only create tables for classes it has actually loaded.

Commit checkpoint

`git add -A && git commit -m 'feat(backend): SQLite models and CRUD' then push.`

6. Hours 4-7 — File Upload & Parsing

Goal: POST /api/ingest accepts multipart file uploads (Markdown and PDF), parses them, splits into chunks, and stores in SQLite.

Prompt 8 — File parsing service

PROMPT TO PASTE INTO YOUR LLM

Create backend/integrations/file_parser.py with these functions:

- `parse_markdown(file_bytes: bytes) -> str`
Decodes UTF-8 and returns the text as-is. Strip leading/trailing whitespace.
- `parse_pdf(file_bytes: bytes) -> str`
Uses pypdf.PdfReader to extract text from all pages. Concatenate page texts with '\n\n' between pages. Handle the case where text extraction returns None for a page (treat as empty string).
- `split_into_chunks(text: str, target_size: int = 350, overlap: int = 50) -> list[str]`
Splits text into chunks of approximately target_size characters. Splits ON paragraph boundaries (double newline) when possible – never split mid-sentence if avoidable. If a paragraph is longer than 2*target_size, split it by sentences (split on '. '). Adds 'overlap' characters from the end of each chunk to the start of the next to preserve context. Strips chunks of leading/trailing whitespace. Discards chunks shorter than 30 characters as noise.
- `detect_file_type(filename: str) -> str`
Returns 'markdown' if filename ends with .md or .markdown, 'pdf' if .pdf, raises ValueError otherwise.

Add a unit test in backend/tests/test_file_parser.py that:

1. Tests parse_markdown with a hardcoded markdown string
2. Tests split_into_chunks produces sensible chunks (right number, no empty ones)
3. Tests detect_file_type for .md, .pdf, and an invalid extension

(Skip the PDF test for now – pypdf needs actual PDF bytes which is annoying to mock.)

Prompt 9 — The /api/ingest endpoint

PROMPT TO PASTE INTO YOUR LLM

Create backend/routes/ingest.py with a router that handles POST /api/ingest.

Endpoint signature:

```
@router.post('/api/ingest')
async def ingest(files: list[UploadFile], db: AsyncSession = Depends(get_db))
```

Behavior:

1. For each file in files:
 - a. Read file bytes (await file.read())
 - b. Detect type via detect_file_type(file.filename)
 - c. Parse: parse_markdown or parse_pdf
 - d. Split into chunks via split_into_chunks
 - e. Create a Document row (source_type='file', source_name=file.filename)
 - f. Create Chunk rows in batch (call create_chunks_batch)
2. After all files processed, return:

```
{
  'status': 'success',
  'documents_imported': <count of docs>,
  'chunks_count': <total chunks across all docs>,
  'document_ids': [<list of doc IDs>]
}
```
3. If a file type is unsupported, skip it (don't fail the whole request) but include the filename in a 'skipped' list in the response.
4. If parsing fails for a file, skip it similarly with 'errors' list in response.

Wire up the router in backend/main.py: app.include_router(ingest_router).

Test it with curl:

```
curl -X POST http://localhost:8000/api/ingest -F 'files=@somefile.md'
```

Show me the curl test output after implementation.

Prompt 10 — Add the BackgroundTasks pattern

PROMPT TO PASTE INTO YOUR LLM

Update the `/api/ingest` endpoint to use FastAPI BackgroundTasks for the embedding/classification step. The flow should be:

1. CRITICAL PATH (user waits): parse files, split chunks, write Document + Chunk rows to SQLite. This must complete BEFORE the response is returned.
2. BACKGROUND (after response): for now, this is a placeholder function that just prints 'Background task: would embed and classify N chunks here'. Person 2 will plug in real embedding/classification later.

The endpoint signature becomes:

```
async def ingest(
    files: list[UploadFile],
    bg: BackgroundTasks,
    db: AsyncSession = Depends(get_db)
)
```

Inside the endpoint, after creating chunks, call:

```
bg.add_task(post_ingest_pipeline, [chunk.id for chunk in all_chunks])
```

Where `post_ingest_pipeline` is a function in `backend/services/post_ingest.py` that currently just prints a message. Person 2 will update this function later – leave a TODO comment: `'# TODO P2: replace with intelligence.embedder.embed_chunks + intelligence.classifier.classify_chunks'`.

This way, the user sees 'Imported 5 documents, 247 chunks' immediately, and the embedding work happens after.

Heads-up

FastAPI's BackgroundTasks runs in the same process as the API. If you have a heavy embedding workload, the API server's CPU is busy until it completes. For 247 chunks with BGE-small, this is ~5-15 seconds — acceptable for the demo. For production scale, you'd switch to Celery + Redis. Don't bother for the hackathon.

Commit checkpoint

`git commit -m 'feat(backend): file ingestion with markdown + PDF support'` then push. At this point, Person 2 can already start consuming chunks from your database — share the `dory.db` file shape with them.

7. Hours 7-12 — Endpoint Shells & Routing

Goal: all 8 endpoints exist as shells returning realistic mock data. Person 3 can develop the entire frontend against these mocks even though Person 2's intelligence isn't ready yet.

Prompt 11 — Mock data fixtures

PROMPT TO PASTE INTO YOUR LLM

Create backend/services/mocks.py with realistic mock data for development.

Functions:

- `mock_health_response(time_offset_hours: float = 0) -> dict`

Returns realistic dashboard data:

```
{
  'categories': [
    {'name': 'technical', 'avg_retention': 0.65 - 0.01*time_offset_hours, 'urgency':
    'medium', 'count': 412},
    {'name': 'personal', 'avg_retention': 0.85 - 0.005*time_offset_hours, 'urgency':
    'low', 'count': 234},
    {'name': 'reference', 'avg_retention': 0.45 - 0.015*time_offset_hours,
    'urgency': 'high', 'count': 189},
    {'name': 'general', 'avg_retention': 0.72 - 0.008*time_offset_hours, 'urgency':
    'low', 'count': 1465}
  ],
  'total_chunks': 2300,
  'time_offset_hours': time_offset_hours
}
(Clamp retention values between 0 and 1.)
```

- `mock_search_results(query: str, top_k: int = 10) -> dict`

Returns 5-10 fake chunks with content, score, retention, source_name, etc.

- `mock_discovery_response() -> dict`

Returns either `{'has_discovery': true, 'chunk': {...}, 'reason': 'You wrote about this 2 weeks ago'}` or `{'has_discovery': false}`, alternating randomly.

- `mock_quiz_questions() -> list[dict]`

Returns 5 fake MCQs: `[{'question': 'What is X?', 'options': ['A', 'B', 'C', 'D'], 'correct_index': 0}]`.

Use Faker library for realistic-looking text (add to requirements.txt: faker).

Prompt 12 — All 8 endpoints as shells

PROMPT TO PASTE INTO YOUR LLM

Create the remaining endpoint shells in backend/routes/. Use mocks.py for now – Person 2 will replace mock calls with real intelligence calls later.

Files to create:

1. backend/routes/health.py – GET /api/health (with optional ?time_offset_hours=N)
2. backend/routes/search.py – POST /api/search (body: {query: str, top_k: int})
3. backend/routes/discovery.py – GET /api/discovery
4. backend/routes/review.py – POST /api/review/{chunk_id}
5. backend/routes/quiz.py – POST /api/quiz/start (body: {category?: str}), POST /api/quiz/submit (body: {quiz_id: str, answers: list[int]})

For each endpoint:

- Use proper Pydantic request/response models (define them in schemas.py)
- Add a TODO comment: '# TODO P2: replace mock with intelligence.X.Y(...)'
- /api/review/{chunk_id} is the only one that does real work – call update_chunk_access
- Return correct HTTP status codes (200 for GET/POST success, 404 if chunk_id not found)

Wire all routers in main.py. After completing, run `uvicorn backend.main:app --reload` and visit /docs to confirm all 8 endpoints appear.

Prompt 13 — End-to-end smoke test

PROMPT TO PASTE INTO YOUR LLM

Write a shell script backend/scripts/smoke_test.sh that uses curl to hit every endpoint and verifies the response shapes. For example:

```
echo 'Testing /api/health...'
curl -s http://localhost:8000/api/health | jq '.categories | length'
echo 'Testing /api/search...'
curl -s -X POST http://localhost:8000/api/search -H 'Content-Type:
application/json' -d '{"query": "test", "top_k": 5}' | jq '.results | length'
...
```

Cover all 8 endpoints. Run with: bash backend/scripts/smoke_test.sh.

Make the server run on port 8000 first, then run the script. All endpoints should return 200 with non-empty JSON.

Hour 12 milestone — handoff to Person 3

At Hour 12, all 8 endpoints work with mock data. Tell Person 3 in chat: *'Backend mock endpoints are live on localhost:8000. Swagger UI at /docs. Build against these — when Person 2 plugs in real logic, the JSON shapes won't change.'*

8. Hours 12-20 — Integration with Person 2's Intelligence Modules

Goal: replace each mock with the real call to Person 2's modules. By Hour 20, the API returns real data computed by real algorithms.

Coordination protocol

Person 2 has been working on /intelligence/ in parallel. Their modules should be ready by Hour 12. Pull their branch (git pull origin feat/intelligence) and verify the imports work. If Person 2 isn't ready yet, keep mocks — they'll catch up.

Prompt 14 — Wire up /api/health

PROMPT TO PASTE INTO YOUR LLM

Update backend/routes/health.py to call real intelligence:

1. Import: from intelligence.decay import compute_retention, aggregate_by_category
 2. In the handler:
 - a. Get all chunks from db: chunks = await get_all_chunks(db)
 - b. If chunks list is empty, return mock data (so empty DB doesn't break demo)
 - c. Call retentions = compute_retention(chunks, time_offset_hours)
 - d. Call categories = aggregate_by_category(chunks, retentions)
 - e. Format the response matching the contract:

```
{
  'categories': [{ 'name', 'avg_retention', 'urgency', 'count'}, ...],
  'total_chunks': len(chunks),
  'time_offset_hours': time_offset_hours
}
```
 - f. Compute 'urgency' from avg_retention: >0.7=low, 0.4-0.7=medium, <0.4=high
- Remove the TODO comment. Add error handling: if intelligence module call fails (Person 2 didn't push yet), fall back to mock_health_response and log a warning.

Prompt 15 — Wire up /api/search

PROMPT TO PASTE INTO YOUR LLM

Update backend/routes/search.py:

1. Import: from intelligence.search import hybrid_search
 2. In the handler:
 - a. Call results = await hybrid_search(query, top_k)
 - b. results is a list of (chunk_id, score) tuples
 - c. Fetch the actual Chunk rows: chunks = await get_chunks_by_ids(db, [r[0] for r in results])
 - d. Compute current retention for each: retentions = compute_retention(chunks)
 - e. Build response:

```
{ 'results': [{ 'chunk_id', 'content', 'score', 'retention', 'source_name',
  'category'}, ...] }
```
 - f. Sort by score (descending), keep top_k
- Same fallback pattern: if hybrid_search isn't available, fall back to mock results.

Prompt 16 — Wire up /api/discovery

PROMPT TO PASTE INTO YOUR LLM

Update backend/routes/discovery.py:

Discovery logic: find a chunk where retention < 0.5 AND it has high relevance to recent activity. For the hackathon, simplify to: find the chunk with the lowest current retention that hasn't been reviewed in the past 24 hours.

```
1. Get all chunks
2. Compute retentions for all
3. Filter to chunks where retention < 0.5 and last_accessed > 24 hours ago
4. If empty list, return {'has_discovery': false}
5. Pick the chunk with lowest retention
6. Return:
{
  'has_discovery': true,
  'chunk': <ChunkOut as dict>,
  'reason': 'You haven't looked at this in {N} days – likely 50% forgotten'
}
Where N = days since last_accessed.
```

Prompt 17 — Wire up /api/quiz

PROMPT TO PASTE INTO YOUR LLM

Update backend/routes/quiz.py:

POST /api/quiz/start:

```
1. Optional body param: category (e.g., 'technical'). If absent, use all chunks.
2. Get chunks (filtered by category if provided)
3. Compute retentions
4. Pick the 5 chunks with the lowest retention (these are the ones the user is
   most likely to have forgotten – perfect for a quiz)
5. Call: from intelligence.quiz import generate_quiz_questions; questions =
   await generate_quiz_questions(chunks)
6. Generate a quiz_id (uuid.uuid4().hex)
7. Cache the quiz in memory: store {quiz_id: {'questions': questions, 'chunks':
   chunk_ids, 'created_at': now}}. Use a module-level dict – don't bother with
   Redis for the hackathon.
8. Return: {'quiz_id': quiz_id, 'questions': [{'question', 'options'} for q in
   questions]}
(Don't return correct_index – that's checked on submit)
```

POST /api/quiz/submit:

```
1. Body: {quiz_id: str, answers: list[int]}
2. Look up the quiz in memory cache
3. Compare answers to the stored correct_indices
4. For each correct answer, also update the corresponding chunk's access (call
   update_chunk_access – answering correctly counts as a review)
5. Return: {'score': N/5, 'correct_count': N, 'detail': [{question_id,
   was_correct, correct_index}]}
```

Prompt 18 — Wire up the post-ingest pipeline

PROMPT TO PASTE INTO YOUR LLM

Update backend/services/post_ingest.py to call Person 2's modules.

```
async def post_ingest_pipeline(chunk_ids: list[str]):  
1. Open a new db session (background tasks need their own)  
2. Fetch the chunks: chunks = await get_chunks_by_ids(db, chunk_ids)  
3. Get embeddings: embeddings = embed_chunks([c.content for c in chunks])  
4. Add to ChromaDB: from intelligence.embedder import store_embeddings;  
   store_embeddings(chunks, embeddings)  
5. Classify: categories = await classify_chunks([c.content for c in chunks])  
6. Update each chunk's category in db: for chunk_id, cat in zip(chunk_ids,  
   categories): await update_chunk_category(db, chunk_id, cat)  
7. Log completion: print(f'Embedded and classified {len(chunks)} chunks')  
  
Same fallback rule: if any P2 module fails, log error and don't fail the whole  
pipeline. The chunk is in SQLite already – partial state is fine.
```

Hour 20 milestone — full data flow

At Hour 20: a user can upload a file, see real chunks ingested, see real retention data on the dashboard, run a real semantic search, get a real discovery suggestion, and take a real quiz. End-to-end working system. The next 12 hours are deployment and polish.

9. Hours 20-30 — Deployment to Render

Prompt 19 — Production-ready FastAPI config

PROMPT TO PASTE INTO YOUR LLM

Make backend/main.py production-ready:

1. Read PORT from env (default 8000) – Render uses dynamic ports
2. Add a /api/keepalive endpoint that returns 'ok' (used to prevent Render free-tier sleep)
3. Conditional CORS: in production (when env var ENV=production), only allow the Vercel URL from FRONTEND_URL env var. In dev, allow localhost.
4. Add a Procfile at project root: 'web: uvicorn backend.main:app --host 0.0.0.0 --port \$PORT'
5. Pin all package versions in requirements.txt (no '>=', use '==')
6. Add a runtime.txt with 'python-3.11.8' so Render uses the right Python version
7. Make sure the SQLite database path is configurable. In production on Render, files in the working dir are ephemeral – but for the demo, that's fine, the database resets on each deploy.

STOP — HUMAN STEP REQUIRED

Now you deploy to Render. This is a manual sequence — the LLM cannot do it for you.

Steps:

1. Push your latest code to GitHub (your feat/backend branch is fine for now)
2. Go to dashboard.render.com
3. Click 'New +' → 'Web Service'
4. Connect your GitHub repo
5. Choose your branch (feat/backend or main if merged)
6. Settings:
 - Name: dory-backend
 - Region: Oregon (or whichever is closest to your judges)
 - Branch: feat/backend (or main)
 - Runtime: Python 3
 - Build Command: `pip install -r requirements.txt`
 - Start Command: `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`
 - Instance Type: Free
7. Add environment variables (Settings → Environment):
 - GROQ_API_KEY = (paste your key)
 - DATABASE_URL = `sqlite+aiosqlite:///./dory.db`
 - ENV = production
 - FRONTEND_URL = `https://<person-3-vercel-url>` (get from Person 3)
8. Click 'Create Web Service' — first build takes 5-10 minutes
9. Once live, copy the Render URL (e.g., `dory-backend.onrender.com`)
10. Test: `curl https://<render-url>/api/health`

Prompt 20 — Keep-warm script

PROMPT TO PASTE INTO YOUR LLM

Render free tier sleeps after 15 minutes of inactivity. Cold start is ~30 seconds, which would be a disaster during the demo.

Solution: a GitHub Action that pings /api/keepalive every 10 minutes.

Create .github/workflows/keepalive.yml:

```
name: Keep Backend Warm
on:
  schedule:
    - cron: '*/*10 * * * *' # every 10 minutes
workflow_dispatch:
jobs:
  ping:
    runs-on: ubuntu-latest
    steps:
      - run: curl -s https://<your-render-url>/api/keepalive
```

Replace <your-render-url> with the actual URL. Commit and push. Verify the Action runs successfully in the GitHub Actions tab.

Prompt 21 — Tell Person 3 the production URL

Drop the production URL in your team chat: *'Backend deployed at <https://dory-backend.onrender.com> — Person 3, set this as your VITE_API_BASE_URL when deploying the frontend.'*

10. Hours 30-42 — Polish, Bug Bash, Demo Prep

Hour 30-34: bug bash with the team

Sit with Person 2 and Person 3 and click through every flow. Every endpoint. Every edge case. Make a list of bugs. Fix them. Common ones:

- CORS issues — Person 3's URL not in allow_origins list
- Missing fields in JSON responses (Person 3's frontend will crash)
- Empty database edge cases (no chunks → division by zero in aggregation)

Prompt 22 — Add error handling middleware

PROMPT TO PASTE INTO YOUR LLM

Add a global exception handler to backend/main.py that catches unhandled exceptions and returns 500 with a helpful JSON error instead of crashing the server. Format:

```
{
  'error': 'internal_error',
  'message': str(exc),
  'endpoint': request.url.path
}
```

Also add structured logging: every request logs method, path, status, duration. Use FastAPI middleware for this – print to stdout (Render captures it as logs).

Hour 34-38: load demo data

Get Person 2's demo dataset generator output (50-100 chunks with realistic backdated timestamps). Upload it through /api/ingest on the production backend. Verify the dashboard shows realistic data with the Time Machine slider showing meaningful changes.

Hour 38-42: buffer time

Don't add new features. Use this for: dry-run demos, fixing the inevitable last-minute bugs, getting sleep, mental prep for judging. If everything works, sit on your hands.

11. Stretch Tasks (if you finish early)

If you're at Hour 24 and everything is working, here are extras:

- **Add `/api/stats`** — total chunks, total documents, last upload time. P3 can show this in a footer.
- **Add SSE for ingestion progress** — instead of waiting for the response, stream progress events as files are parsed.
- **Add a `/api/admin/reset` endpoint** — clears the database. Useful for the demo (start fresh between dry runs).
- **Add request rate limiting** — slowapi library, 100 req/min per IP.
- **Add file-type validation** with size limits — reject files over 10MB or non-text MIME types.

12. Descope Ladder (if you fall behind)

If you're behind schedule, cut features in this order — top of the list goes FIRST:

#	Cut	What you keep	Demo impact
1	Quiz mode entirely	All other features	One less feature, but core works
2	PDF parsing	Just markdown files	Demo loads .md files only — fine
3	BackgroundTasks	Synchronous embedding (user waits)	5-15 sec delay on upload — annoying but works
4	Production deployment	Localhost demo with screen share	Less impressive but functional

13. Handoff Checklists

Hour 4 → tell P2: 'Database schema is ready, here's the shape'

- Share dory.db file or schema SQL
- Confirm Pydantic ChunkOut shape matches contract
- P2 can now write decay/embedder modules against this shape

Hour 12 → tell P3: 'All endpoints live with mock data'

- Production URL? No — localhost:8000 for now
- Swagger UI: localhost:8000/docs
- All 9 endpoints return realistic JSON
- P3 can build entire frontend against these mocks

Hour 20 → tell team: 'Real intelligence wired up'

- /api/health returns real decay-computed retentions
- /api/search uses real BM25 + dense + RRF
- Quiz generation uses real Groq calls

Hour 24 → tell P3: 'Production URL is X — update your env'

- Render URL after first successful deploy
- P3 redeploys frontend with new env var

14. Troubleshooting Guide

Symptom	Likely cause	Fix
'Address already in use' on port 8000	Previous uvicorn process still running	lsof -ti:8000 xargs kill -9 (macOS/Linux) or restart terminal
CORS error in browser console	Person 3's URL isn't in allow_origins	Add their URL (localhost:5173 or Vercel URL) to CORSMiddleware
SQLAlchemy 'no such table'	init_db() didn't run, or models weren't imported	Verify model imports in models/__init__.py; rerun server
Render deploy fails on 'pip install'	Missing package or version conflict	Check Render build logs; try locally with same Python version
Render shows 'application exited'	Crash on startup — check logs	Likely missing env var. Verify all required vars are set in Render dashboard
Cold start takes 30+ seconds	Free tier sleep — expected behavior	Keep-warm GitHub Action (Hour 24). Or wake it up before judging by hitting /api/health
BackgroundTask never runs	Maybe the task crashed silently	Add try/except + print in the task. Errors don't propagate to the response
Pytest 'no module named backend'	Python path issue	Run from project root: python -m pytest backend/tests/

Person 1 Build Guide — Backend & Integration

Last updated April 2026

If something is wrong or unclear, ask in team chat. Don't suffer alone.